

Lab Report: Graphs and Graphing Algorithms

Course: 2XC3 - Algorithms and Data Structures

Prepared By:

Ben Guo
Heeman Ijaz
Jason Xie

Submitted To:

Dr. Ilir Dema
Department of Computing and Software
McMaster University

February 20, 2026

Contents

Executive Summary	1
1 Experiment 1: Probability of a Cycle in an Undirected Graph	2
1.1 Objective	2
1.2 Methodology	2
1.3 Results	2
2 Experiment 2: Probability of Connectedness in an Undirected Graph	4
2.1 Objective	4
2.2 Methodology	4
2.3 Results	4
3 Approximation Experiments	6
3.1 Objective	6
3.2 Methodology	6
3.2.1 Approximation Method 1	6
3.2.2 Approximation Method 2	6
3.2.3 Approximation Method 3	6
3.2.4 Experiment 1 Method	7
3.2.5 Experiment 2 Method	7
3.2.6 Experiment 3 Method	7
3.3 Results	7
3.3.1 Experiment 1 Results	7
3.3.2 Experiment 2 Results	9
3.3.3 Experiment 3 Results	10
4 Independent Sets and Vertex Cover Analysis	13
4.1 Objective	13
4.2 Methodology	13
4.3 Results	14
A Code Navigation Guide	16

List of Figures

1	probability of cycle vs edge-to-node ratio as # of nodes grows	3
2	probability of connectedness for 10, 50, and 100 nodes relative to edge/node ratio	4
3	approx. ratio relative to number of edges	8
4	approx. ratio relative to number of nodes	9
5	visualization of worst case scenario for greedy algorithm	11
6	approx. ratio frequency for 5 node graphs	11
7	MIS vs MVC size frequency on a dense graph	14
8	MIS vs MVC size frequency on a sparse graph	15

Executive Summary

In this lab, we implement, test, and analyze graph theory algorithms and properties. First, we implemented BFS and DFS variations as well as cycle and connectedness detection algorithms. Then, we tested these implementations on random graphs trying to find a link between the parameters of a graph and the probability of cycles/connectedness. Next, we implement approximation methods for finding the vertex cover (VC) of a graph and test the algorithms to analyze their behavior, mostly compared to finding the minimum vertex cover (MVC) of the graph and their accuracy. We then implement brute force methods for finding the maximum independent set (MIS) of a graph, and the intrinsic relationship of the MIS with the MVC for any graph, as well as what these two sets actually tell us about a graph.

Key Takeaways

- **Understanding the difference between BFS & DFS:** Despite both being graph traversal algorithms, both have unique behavior and applications, and we use those behaviors to implement 2 different algorithms for finding different properties of a graph
- **Finding and analyzing properties of an algorithm:** We discover a relationship with the number of nodes and the number of edges, relative to the connectedness & existence of a cycle in the graph
- **Using approximations to find vertex covers:** Find accurate MVCs is extremely difficult for large graphs due to being NP-hard (no polynomial solution) so we test different approximation methods and their behavior both to better understand what VC means, and how we can approximate for large graphs and the use cases for different approximation methods
- **The MIS and MVC relationship:** Finding the MIS and MVC of a graph through brute forcing is quite similar, just with the logic flipped. This gives the hint we need to decipher their relationship, and how they are essentially complements of each other with respect to a graph

1 Experiment 1: Probability of a Cycle in an Undirected Graph

1.1 Objective

Our goal is to understand what the probability of a cycle in a graph based on the number of nodes and edges, i and j .

1.2 Methodology

There was 2 stages of testing, first testing values to show how we carry out testing, and another stage to actually do the testing. First, we wanted to understand the relationship between nodes and edges, so my first idea was to use a ratio as they scale. We can have a few fixed number of nodes, and 'slide' the number of edges as a ratio relative to the nodes to scale. For each, we run 1000 trials and then find the percentage of graphs of that type that contained a cycle. We then plotted the probability of a cycle vs the ratio of edges to nodes, and we split the possible ratios between 0 to 1.5 with 30 equal steps using numpy's linspace function. We noticed a limit that we approached, at a ratio of about 0.5, so we graphed that too for testing.

1.3 Results

The most firm conclusion I can draw is that as the number of nodes approaches infinity, at a ratio of 0.5 the probability of a cycle reaches 100%. There are a few things to unpack here. First, it is clearly hard to draw any conclusions when the number of nodes are low, i.e., < 1000 . The main reason for this is the randomness, which is displayed in Figure 1. Even with thousands of trials, when there are only 10 nodes, it is reasonably possible to have only graphs with cycles or only graphs with no cycles for many edge ratios. As we scale the number of nodes, we reduce the likelihood of outliers affecting the sample data, and so we look at graphs with large numbers of nodes.

Our goal is to see when the probability hits 100% what do we know about the graph, and that is why we went with this conclusion. The exact number of nodes is actually not relevant, just that we have enough. From this, we see that if we have about 1 edge for every 2 nodes, the chance of a cycle is almost 100%. Logically, this makes perfect sense: 1 edge is composed of 2 nodes, i.e., if there is 1 edge for every 2 nodes then the average degree of a node is 1. In other words, on average every node has 1 outgoing edge. Then we can make the connection: if every node has 1 outgoing edge, then at a large enough size it makes sense that we are basically guaranteed a cycle. This is less likely with less nodes, even if there are many edges there is always the chance one node gets many more edges. It is just probability, if we make 10 draws, an outlier heavily skews the data, but by 5000 draws these outliers begin to cancel out.

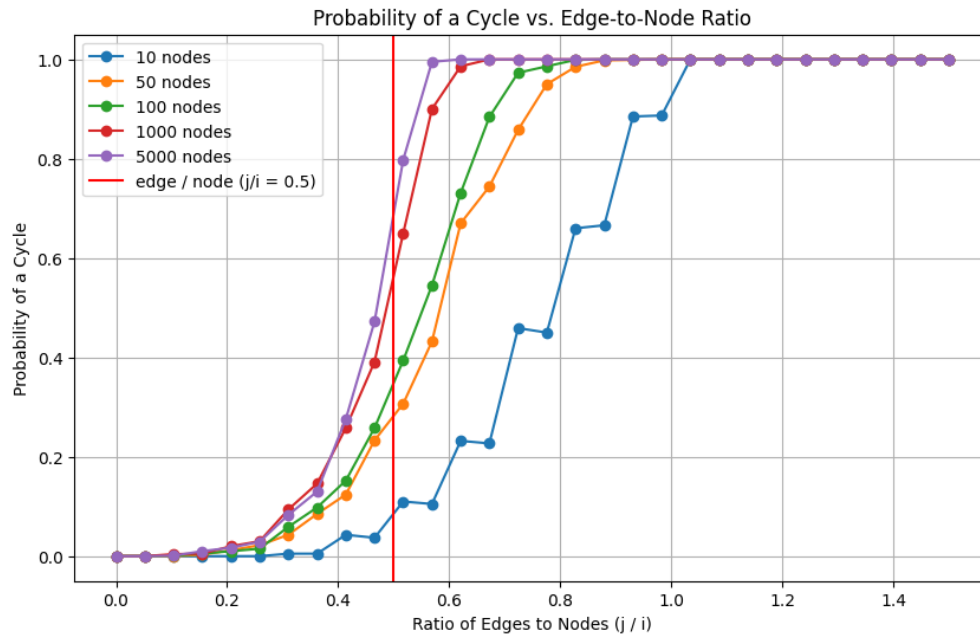


Figure 1: probability of cycle vs edge-to-node ratio as # of nodes grows

2 Experiment 2: Probability of Connectedness in an Undirected Graph

2.1 Objective

Our goal is similar to the previous experiment, however this time we are looking for the connection between connectedness and the nodes and edges.

2.2 Methodology

Since our goal is similar, our methodology will also be similar. This time, we will observe edge/node ratios from 0 to 4 split into 40 intervals, since having a fully connected graph could take potentially many more edges than having a cycle, and we need a wider range to see this change. We set our nodes to 10, 50 and 100 as that gives us more than enough information to work with, and fits best in our parameters (i.e., 4.0 edge/node ratio). For each discrete combination of nodes to edges, we ran 5000 trials to smooth out noise.

2.3 Results

This result is quite unique. When observing connectedness, there was a sort of 'threshold' ratio at which all graphs jumped to 100%, telling us connectedness is dependent on this ratio. However, here we find something different: The connectedness depends on the number of nodes. As Figure 2 shows, if there are a few nodes (i.e., 10) then by 2 edges per node, we most likely have a connected graph already. However, the same ratio for 100 nodes only has a 20% chance of being connected.

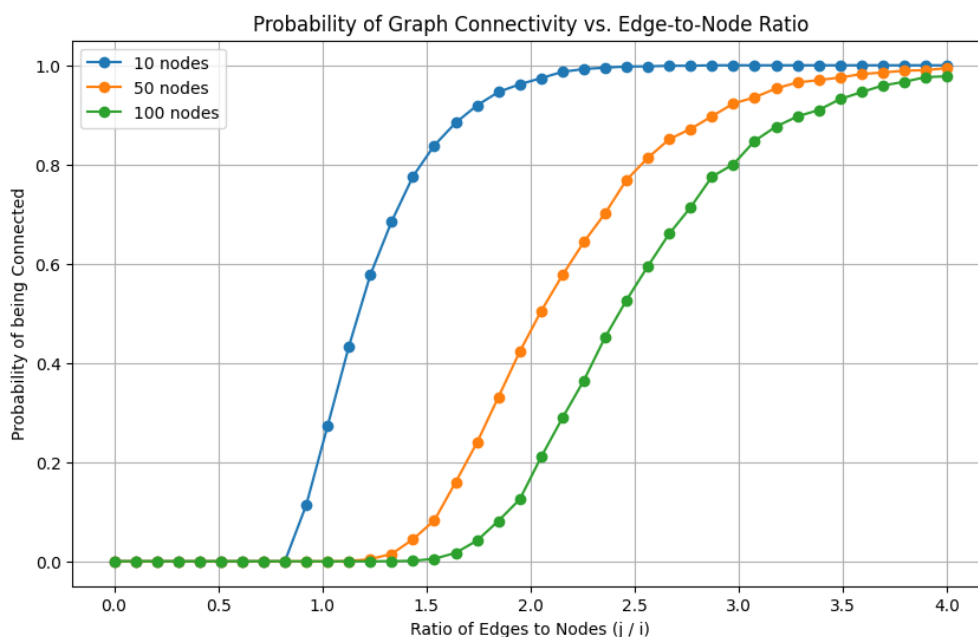


Figure 2: probability of connectedness for 10, 50, and 100 nodes relative to edge/node ratio

A graph becomes fully connected the moment the last isolated node is attached to the rest. This means that in a scenario with n nodes, even if the first $n - 2$ edges perfectly attach $n - 1$ nodes, it is possible that we add the maximum possible edges to all other nodes before including this n th node. This is the logic that gives us this behavior, where the probability curve has a rightward shift as the number of nodes grows. For a small graph, the odds are much lower that one node is avoided, and the number of wrong edges are less in total, before we must connect the last node. This is not the same for graphs with many nodes, where our curve starts to get 'pushed down', as we need a higher ratio to increase the probability every node is connected, we need more 'draws' to guarantee the last node is picked as an edge.

From some simple math, we can see that the total pool of edges to choose from is $\binom{n}{2}$, or $\frac{n!}{2(n-2)!} = \frac{n(n-1)}{2}$, which is roughly quadratic in growth relative to the number of nodes. However, the number of edges that connect the final isolated node is always $n - 1$, meaning that as the number of nodes grows, the number of edges to draw from increases quadratically, but the number of 'correct' edges for the final node grows linearly, showing the shift in Figure 2 for larger graphs.

3 Approximation Experiments

3.1 Objective

Our goal is to investigate the 3 implementations that approximate the MVC of a given undirected graph G . We will analyze how they work and what their behavior tells us about finding MVCs, and the different techniques we use to do so.

3.2 Methodology

First, we will discuss the three approximation methods in brief and explain how they work. Then we will outline the reasoning behind the three experiments we will run and how they work.

3.2.1 Approximation Method 1

Our first method can best be described as a 'greedy' approach. We target the node with the most connections, and add it to our VC, removing it from the original graph and continuing until there are no edges left that do not connect to a node in our VC. The idea is quite intuitive, since we would assume creating a minimal set of nodes connecting every edge would favor nodes connected to many edges. Of course, there is the possibility where greediness can allow us to take a node with many connections when 2 nodes with required connections would suffice (i.e., we take the 'best' node but end up requiring the 2 'lesser' nodes as well, when we could have accomplished the same with only the 2 'lesser' nodes, 2 $\leq$ 3). We will investigate this further in depth in the 'Results' section.

3.2.2 Approximation Method 2

This method is the wildest of the three: We will randomly pick nodes until our set is a vertex cover. There is no logic, and as such it is extremely hard to pin down how it performs, especially in comparison with the other 2 algorithms. Theoretically, we could pick the n nodes that make up the MVC in our first n choices and are done. However, we could also randomly pick all other nodes before we reach the 1 node that is **required** to connect all edges, resulting in the largest possible VC. That is the inherent problem with randomness, and also its greatest strength.

3.2.3 Approximation Method 3

This method uses randomness, similar to the previous algorithm, albeit in a much more controlled manner. We pick a random edge and add both nodes that define the edge to our VC, removing the nodes from the graph. We continue until we have no edges left. This makes it difficult to guarantee a MVC, especially compared to the other 2 methods, since every edge means we have to add 2 nodes. However, it also guarantees that our VC will be, at worst, 2 times the size of the actual MVC. This is clear, since each edge we check will require **at least** 1 of the nodes to be added, which means at **worst** we are adding twice as

many nodes for each step (of course, it is possible BOTH nodes need to be added eventually, which is where we have the best case for this method for each addition).

3.2.4 Experiment 1 Method

Since a VC is fundamentally dependent on the edges of a graph, we can test random graphs with a fixed number of nodes varying from 1 edge to the maximum possible edges. We choose the size of the graph to be 10 nodes, which gives us a wide sample size, and also lets us calculate the MVC in a reasonable time frame. Then, since the maximum possible edges for a graph with 10 nodes is 45, we will test graphs with all possible edges between 1 and 45, running 1000 trials per graph to remove noise. Then, we will compare the size of the VC for each approximation to the MVC, and graphing the ratio relative to edges.

3.2.5 Experiment 2 Method

Now that we know the relationship with edges, we can also test the performance as the size of the graph grows but maintains the same relative density. For the density, I settled on 40%, enough edges to give the algorithms some complexity, while not being dense enough to allow the algorithms to have an easy time (explained further in results). In fact, this density was chosen due to the results of the previous experiment, showing the density where the graphs performed most consistently was around this mark. So, we iterated from 4 nodes to 13 nodes and generated a thousand random graphs for each to average out. 4 nodes guarantees we have at least a few edges, while a max of 13 nodes is complicated enough while being computable. This time, we graph the ratio relative to the number of nodes.

3.2.6 Experiment 3 Method

Finally, to analyze the behavior of the algorithms in the worst situations, we will check every possible graph with 5 nodes, which will give use 1024 possible graphs. 6 nodes would give us 2^{15} graphs, which is a little too many, and any less nodes than 5 would not provide us with enough data. So, we will generate every possible graph, and check the approximation ratio for each method, and when we find a ratio for each method that is the worst so far, we will keep track of the ratio and the graph on which it performed the worst. Then, we will also track the exact frequency of ratios for each algorithm and graph them.

3.3 Results

We will analyze the results experiment by experiment and finally draw conclusions about the approximations and vertex covers in general

3.3.1 Experiment 1 Results

This experiment is a good starting point, and gives us a good amount to go off of regarding the behavior of all approximations. First, Figure 3 shows that the overall trend is that having more edges makes each approximation more accurate, in other words the density of the graph in terms of edges directly correlates with the overall accuracy. In fact, at maximum density

(i.e., every node connects to every other node) we have perfect accuracy for the first two approximation methods. The first method picks the best next option, and if the connections between every node is identical, no matter our order we will always pick the optimal MVC of size $n - 1$. The same is true for the second approximation, where we pick randomly and always check if we have succeeded, which is guaranteed after $n - 1$ nodes. We actually do not have the most optimal result even at max density for approx. 3. In fact, we know our approx VC is 10, and the optimal MVC is 9, giving a ratio of $1.1\bar{1}$ since we add 2 nodes at a time, and always end up over counting. If we ran this experiment with an odd number of nodes, we would actually have the perfect ratio at 100% density for this algorithm too!

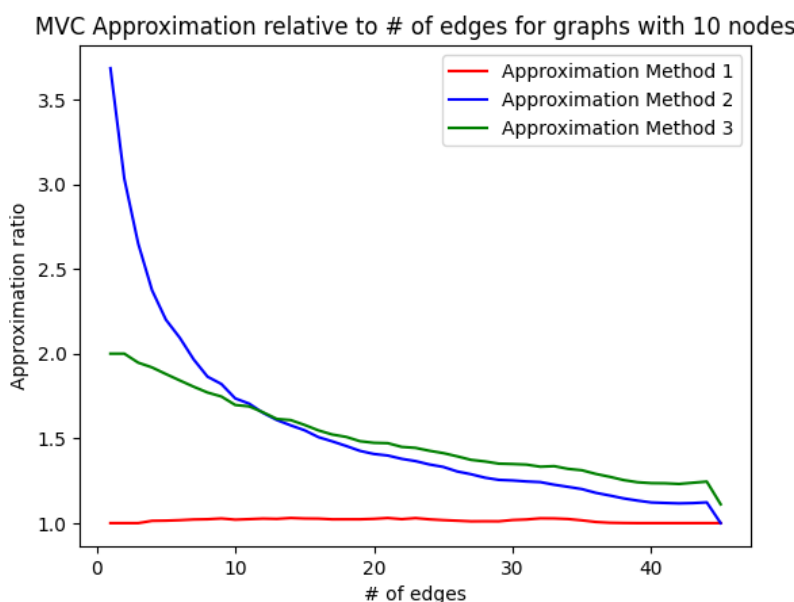


Figure 3: approx. ratio relative to number of edges

Before considering the improved behavior at higher density, we will briefly cover the performance of the greedy algorithm (1st approximation method). This is slightly misleading, given that our trials might show that this method is far and away the best. The truth is that we do not have the conditions for the worst performance of this algorithm, and the odds of seeing it are canceled by the thousands of trials we run. The greedy algorithm can suffer extreme degradation, especially as the size of the graph in nodes grows, which would increase the odds that being greedy causes a fatal mistake, or multiple. We cannot really test this method well here since we do not have the ability to efficiently calculate the MVC for comparison for extremely large graphs. However it does also show that greedily picking the best choice is extremely efficient and consistent for small graphs, as we will see later on.

The following applies to all methods: in general, with a higher density we would naturally have a larger MVC (since more nodes might be needed to cover every edge). This means the error margins continuously shrink, since if our MVC is around 8 nodes, having a worst case where the random nodes cause us to choose a VC of 10 nodes is much less harmful than, say, if we had a MVC of 4 nodes and we accidentally chose a VC of 6 nodes (even though

both have a similar error of 2 nodes, the ratio is much worse in the latter). Also, the room for error from a MVC with 8 nodes is only 9 or 10 nodes, while for a MVC with 4 nodes, we could have anywhere from 5-10 nodes in our VC.

The behavior on the left side of the graph is also interesting. For one, we continue to show that our 3rd method guarantees a worst case performance of 2 times the MVC size. It shows a noticeable (though not drastic) improvement with density, but its best feature is of course that its ratio is always at worst 2.0 and NEVER worse. In fact, it shines when comparing it to approx. method 2 with extremely sparse graphs. In a sparse graph, the size of the MVC will be extremely small (less edges = less nodes needed to connect them), but the random method does not know this and picks at random. Almost every time, we will not pick the few nodes needed but instead pick a random assortment of useless nodes. Of course, there is the chance we pick exactly what we need right away, but that is sanded away with 1000 trials and we see the average behavior (some lucky, most unlucky).

3.3.2 Experiment 2 Results

We now understand the impact of edges on these algorithms, which corresponds to the sparsity of a graph. Now, we observe the changes based on the actual **size** of the graph. We want to see the impact of nodes on a graph, though we cannot really use the same method of locking edges and having nodes be variable since in this case we would be repeating the same experiment, just testing density in a different way. So, like explained in the methodology section for this experiment, we fixed the **density** of the graph and scaled the nodes.

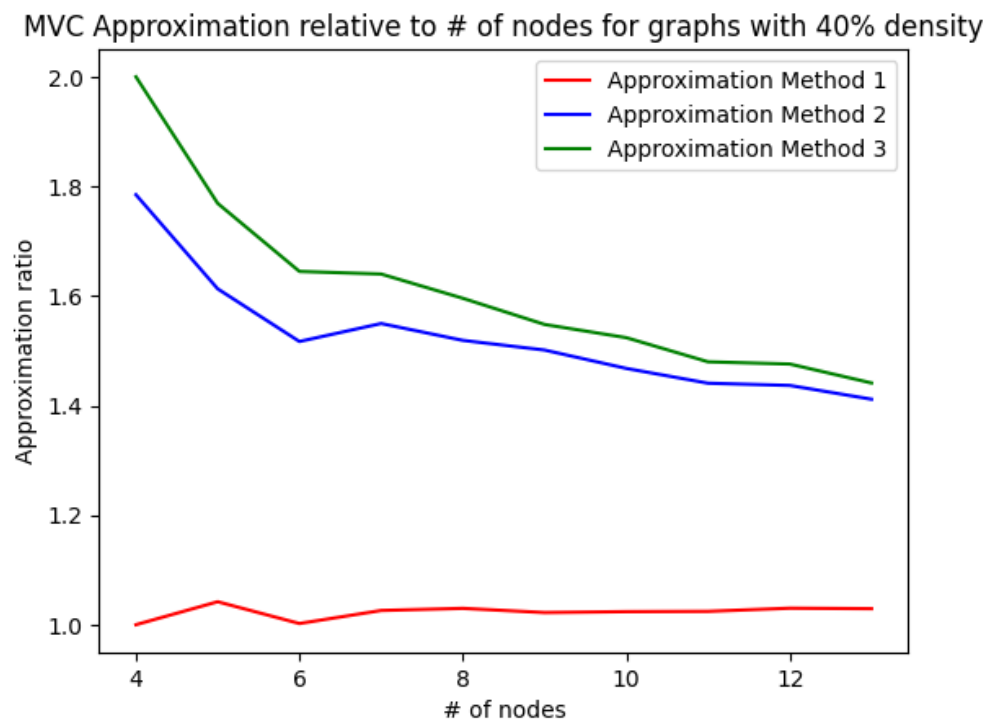


Figure 4: approx. ratio relative to number of nodes

Once again, our greedy algorithm maintains its behavior since just like before we rarely have a worst case situation. The more interesting behavior comes from the other two algorithms, which seem to be improving substantially as the size of the graph grows, even when the relative number of edges is the same. This tells us something important: The size of the actual MVC grows proportionally, in other words, the 'margin of error' we mentioned previously still holds in a different way. Instead of a fixed number of nodes which guarantees our MVC size will approach this number, we continue to increase the number of nodes but still see an increase in accuracy! This more specifically tells us that as the number of nodes increases, even though we make more total mistakes, **the penalty of making a mistake decreases**. At 4 nodes, 40% density is about 2 edges. It is highly likely that the other two methods will make a major mistake by adding an unnecessary node or 2, which is a huge penalty when the MVC is actually only 1 or 2 nodes. However, at 12 nodes, 40% density is about 26 edges, meaning in most cases the MVC will be larger by default, and making even more mistakes than before will still not be as costly.

Both these algorithms work on random chance (random node chosen, random edge chosen) and so mistakes are likely, but as the penalty of a mistake decreases, the methods look much more accurate (even though they actually are not at all). It also means their behavior will be more successful in general, since we are not creating specific outlier cases. When approximation 3 first adds the two nodes of an edge to the VC, we have an extremely high chance of having made the right choice. The odds decrease as we go, but when we do inevitably choose the wrong node, we have a small bump in the ratio. Twice as many incorrect nodes is not a big deal, especially when the total nodes are thrice as many, and the extremely unlucky cases are balanced out by the lucky cases of a perfect MVC over thousands of trials.

3.3.3 Experiment 3 Results

The final experiment provides the most mathematically certain results so far. We tested with every possible 5-node graph, and so the only remaining random element is the random node/edge selection of methods 2 and 3. Again, we can start with method 1, which has the clearest results so far. In almost every graph, we have a perfect approximation ratio, except for 1 single case where the greedy algorithm fails (1.5 ratio), and for that, we have the graph structure too:

```
— Graph Structure (Nodes: 5) —  
Node 0 -> [4]  
Node 1 -> [3]  
Node 2 -> [3, 4]  
Node 3 -> [1, 2]  
Node 4 -> [0, 2]
```

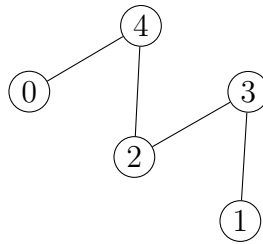


Figure 5: visualization of worst case scenario for greedy algorithm

This gives us the insight we need into the truth behind the greedy algorithm, and why it doesn't perform as well as we might have thought. The optimal choice is clear, we pick 4 & 3, which is a MVC of only 2 nodes. However, by stepping through our logic, we find that our greedy algorithm has to make a decision on whether to pick 4, 2, or 3, since all these nodes have the same number of edges. We make an unlucky mistake, and pick 2 first. Then, we have 2 separate clusters, and we need 2 more nodes to complete our VC and end with 3 nodes, a ratio of $3/2$ or 1.5 . This is the fundamental issue, we have a case where greedy should work, but since we cannot understand the best choice in a tie, we end up creating a huge mess. There are a few more things to understand though. First, we see even in the absolute worst case our greedy algorithm only suffers a slight penalty. This algorithm is punished far less for its mistakes when there are less nodes, but of course the likelihood and consequence of making greedy mistakes grows extremely fast, and in the worst case can be extremely harsh. Of course, we could change the logic to work for this graph, but then another issue will pop up elsewhere, and like a game of whack-a-mole we cannot expect to remove all errors to make the greedy algorithm perfect.

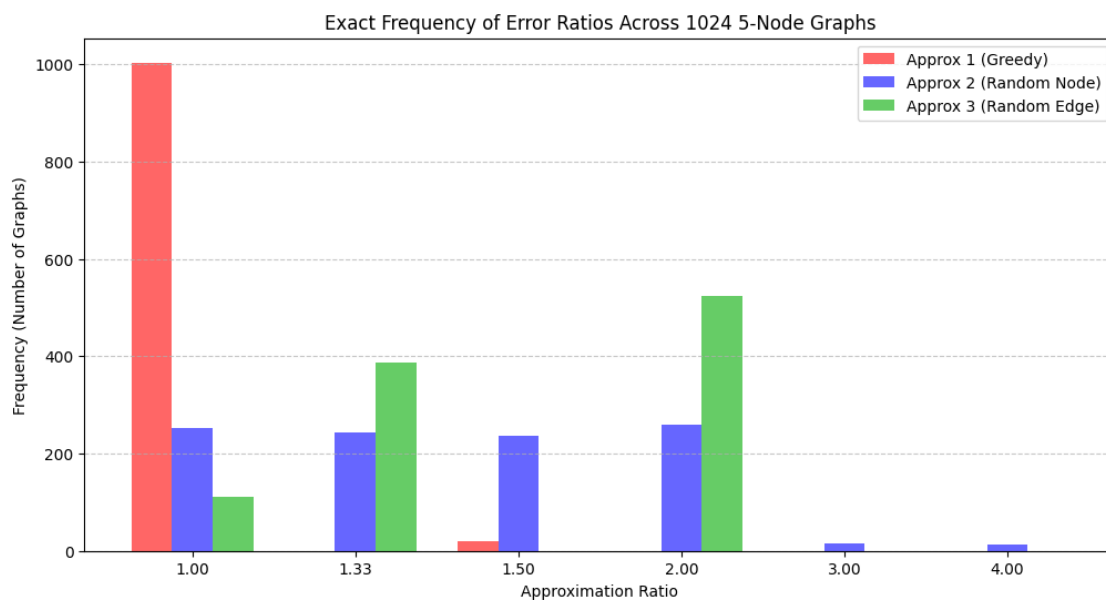


Figure 6: approx. ratio frequency for 5 node graphs

The second method reiterates a lot of the points we covered earlier. For one, it is the only method to have a worst case that reaches > 2.0 . However, even these worst cases are extremely rare. That makes sense since we are working with 5 node graphs. Think about what a worst case of 4.0 means: Not only must the MVC be much smaller, i.e., 1 node, but our random method must randomly pick all other nodes. But this shows that that is not impossible; out of 1024 graphs, the subset that has only MVCs of size 1 is still large enough that in a few unlucky cases, we pick the worst possible choice every time. That is the nature of randomness, and the almost identical distribution between 1.0 - 2.0 also shows the nature of this randomness. We sometimes get a perfect strike, sometimes we do a little worse, but generally our performance is in the range of 1.0 to 2.0.

The final method demonstrates why we consider it to be the 'safest' algorithm, but not a smart one. Of course, for such a small graph consistently achieving 2.0 ratio is not as impressive when the other algorithms more consistently have better ratios. We can guarantee we won't pass 2.0, but we hit the worst case of 2.0 many times. Essentially, this method shows us what we must trade in efficiency for security. Of course, as we remember, the worst case for this algorithm is when we add pairs of nodes repeatedly, we may have the worst case scenario that we add 2 nodes for each edge, when we could have just done 1 node.

4 Independent Sets and Vertex Cover Analysis

4.1 Objective

Our goal is to first understand and define a function for finding the maximum independent set (MIS), the largest set of nodes for a graph such that no 2 nodes in our set form an edge in the graph. We use brute force, then compare it to our MVC results on a graph and define the relationship between the two through empirical analysis. We also analyze what the density/size of the graph entails for MVC and MIS.

4.2 Methodology

We used the same logic as we did in our MVC implementation, which comes with the same benefits and limitations. Although generating the whole power set is computationally heavy at $O(2^n)$, it provides a simple, correct solution to solve this NP-Hard problem, and for our purposes this is enough. We brute force by checking every single possible power set (all possible subsets of nodes in the graph), and if the set is an independent set and larger than our largest so far, we save it.

```
def MIS(G):
    nodes = [i for i in range(G.number_of_nodes())]
    subsets = power_set(nodes)
    max_cover = []
    for subset in subsets:
        if len(subset) <= len(max_cover):
            continue
        if is_independent_set(G, subset):
            if len(subset) > len(max_cover):
                max_cover = subset
    return max_cover
```

To compare our results with the MVC for a graph, we can run an analysis on thousands of randomly generated graphs and look for a relationship. Since there seemed to be a relation for each with the complement of the other, we decided to graph the size of the MVC vs MIS, along with the frequency of size results. The reasoning behind choosing size to measure is because it is the easiest to graph, and also the fundamental definition of an MVC/MIS, since even though an MVC or MIS can be made of different nodes for the same graph, the size will always be the same.

We decided to analyze two scenarios: a dense graph and a sparse graph. These two definitions depend on the density, i.e., number of edges, so this is the best way to get results. In terms of the exact values, we chose 10 nodes for a good selection while maintaining our $O(2^n)$ complexity for brute forcing both MVC and MIS. The dense graph was populated with 85% of the maximum edges, and the sparse graph is populated with 15% of the maximum edges. Overall, we ran 1000 trials for each to get a good spread of results.

4.3 Results

Our results are extremely interesting. Right off the bat, we know for sure what the relationship is between the MVC and MIS of a graph G : Figure 7 and 8 both show $|MVC| + |MIS| = |V|$, where $|V|$ is the number of nodes in G . Since there might be multiple optimal sets of the same size, a randomly chosen MIS and MVC of a graph may have overlap. However, by their structural relationship, for any MIS, the complement is also a valid MVC, and for any MVC, the complement is also a valid MIS. Of course, a minimum vertex cover contains the least number of nodes that touch every edge, while a maximum independent set contains all nodes where an edge cannot be created by picking any 2. If a node 'u' is in a minimum vertex cover, it necessarily touches 1 or more edges, but all the nodes touched by an edge that is connected to 'u' are automatically safer, since it 'frees' the other nodes to be added to the MIS without worrying about a collision (of course, barring another edge connecting to these nodes).

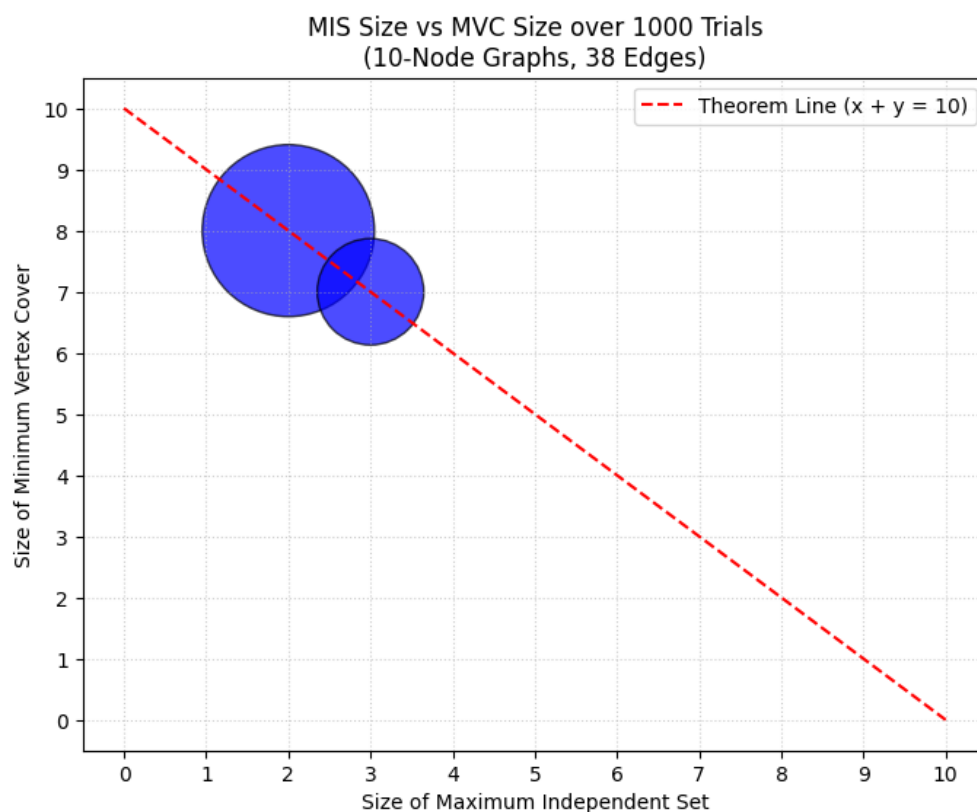


Figure 7: MIS vs MVC size frequency on a dense graph

In this way, the two sets are mathematical inverses, and that is shown in our graphs. While this is the most important result, there is also another result that is quite interesting. It further exemplifies the inverse relationship between the two sets and explains the meaning of each set further. We graphed the frequency of an occurrence by the size of the blue circles. For example, in Figure 7 we have a large blue circle centered on (2, 8) and a smaller circle

centered on (3, 7). Out of 1000 trials for this 10 node dense graph with 38 edges, we only had 2 results; either the MVC was 8 nodes and the MIS was 2 nodes, or the MVC was 7 nodes and the MIS was 3 nodes. This checks out: if the graph has more edges, the ability to choose nodes that are completely independent is quite difficult, while a vertex cover will require more nodes to account for all edges. It is interesting to see that we do not have any sets with more than 8 nodes in the MVC and none with less than 3 in the MIS. Mathematically, these display the limits of our graph and what necessarily must hold for the sizes of both.

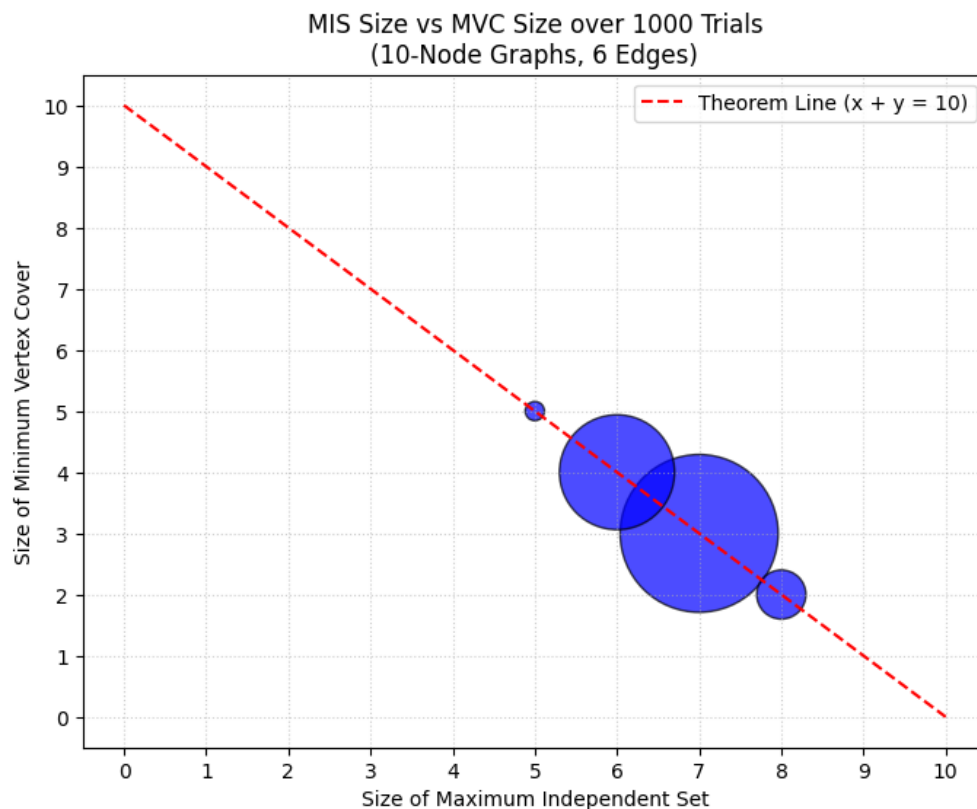


Figure 8: MIS vs MVC size frequency on a sparse graph

We can also observe the trends in figure 8: We have a sparse graph with 10 nodes and only 6 edges, so of course we have a case where we could have a graph with 5 disconnected pairs of nodes, but this is a worst case scenario and not quite common, evidenced by the small size of the frequency circle. The other few scenarios are more common, most likely 3 nodes to create the MVC and 7 nodes free to create the MIS. With fewer edges, we are more likely to have many nodes with no attachments by virtue of randomly generated graphs. However, something quite interesting is that while it is theoretically possible to have 1 node with 6 edges, resulting in (9, 1), we were unable to see this because we only ran 1000 trials, another quirk of using these randomly generated graphs where it would be extremely rare to generate this type of graph. In fact, this even ends up strengthening our results, since mathematically it makes sense for such a rare occurrence to not happen.

A Code Navigation Guide

To verify the results presented in this report, please refer to the attached Jupyter Notebook files. All code was run on a Python 3.14+ kernel. **!!BEFORE RUNNING!!** You must have matplotlib and numpy installed on the kernel before running any of the notebooks. This can be done by running ‘pip install matplotlib’ and ‘pip install numpy’ in the environment. Here is a guide on what is attached:

- **experiments/auxiliary/**: Contains the original graph.py file provided, with implementations of BFS2, DFS2, BFS3, DFS3, has_cycle, and is_connected. It also contains helper.py, an extra python file used in testing/experiments that contains helpful functions such as creating random correct graphs given a number of nodes and edges as well as printing out graphs in an easy to understand way.
- **experiments/**: Contains the main code for the first 2 experiments in jupyter notebooks, as well as the approximation implementations in a python file, 3 approximation experiments (all in 1 notebook), and independent set experiments
- **img/**: Contains all relevant graphs from each experiment, grouped with the respective experiment